

Yolo

Yolo8

```
!pip install ultralytics
!wget -O fire_and_smoke_yolov8.pt
https://github.com/Lefyd24/Forest-Fire-Smoke-Detection/raw/main/models/yolov8/weights/fire\_
model.pt
```

```
import cv2
import numpy as np
from ultralytics import YOLO
from google.colab.output import register_callback, eval_js
from IPython.display import display, Javascript
import base64
```

```
# 1. Load the fine-tuned Fire & Smoke weights file we downloaded
model = YOLO('fire_and_smoke_yolov8.pt')
```

```
# 2. Python backend frame receiver function
```

```
def process_frame(img_data):
```

```
    try:
```

```
        if not img_data:
```

```
            return ""
```

```
        # Decode base64 image data payload arriving from your browser
```

```
        img_bytes = base64.b64decode(img_data.split(',')[1])
```

```
        img_array = np.frombuffer(img_bytes, dtype=np.uint8)
```

```
        frame = cv2.imdecode(img_array, flags=cv2.IMREAD_COLOR)
```

```
        if frame is None:
```

```
            return ""
```

```
        # Run inference against the fire model
```

```
        # The model usually has classes like: 0: fire, 1: smoke
```

```
        results = model(frame, conf=0.35, verbose=False)
```

```
        annotated_frame = results[0].plot()
```

```
        # Encode back to JPEG to display smoothly back inside the browser window
```

```
        _, encoded_img = cv2.imencode('.jpg', annotated_frame)
```

```

        base64_output = 'data:image/jpeg;base64,' +
base64.b64encode(encoded_img).decode('utf-8')
        return base64_output
    except Exception as e:
        return ""

```

```

# Bind our Python processor cleanly into the notebook runtime environments
register_callback('notebook.process_frame', process_frame)

```

3. JavaScript Front-End to drive the loop using the browser context

```

js_stream = Javascript("""
    async function startStream() {
        const div = document.createElement('div');
        const video = document.createElement('video');
        const canvas = document.createElement('canvas');
        const img = document.createElement('img'); // Destination display wrapper
        const stopBtn = document.createElement('button');

        stopBtn.textContent = 'Stop Stream';
        stopBtn.style.display = 'block';
        stopBtn.style.margin = '10px 0';
        stopBtn.style.padding = '5px 10px';
        stopBtn.style.background = '#d9534f';
        stopBtn.style.color = 'white';
        stopBtn.style.border = 'none';
        stopBtn.style.borderRadius = '4px';

        document.body.appendChild(div);
        div.appendChild(stopBtn);
        div.appendChild(img);

        // Connect device video tracks
        const stream = await navigator.mediaDevices.getUserMedia({ video: { width: 640, height:
480 } });
        video.srcObject = stream;
        await video.play();

        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;
        const ctx = canvas.getContext('2d');

        let keepStreaming = true;
        stopBtn.onclick = () => {
            keepStreaming = false;

```

```

        stream.getTracks().forEach(track => track.stop());
        div.remove();
        console.log("Webcam released.");
    };

    // Self-paced execution thread managed directly by your browser window manager
    async function captureLoop() {
        if (!keepStreaming) return;

        ctx.drawImage(video, 0, 0);
        const dataUrl = canvas.toDataURL('image/jpeg', 0.65); // Compressed slightly to run fast

        // Invoke the backend Python engine asynchronously
        const processedDataUrl = await
google.colab.kernel.invokeFunction('notebook.process_frame', [dataUrl], {});

        if (processedDataUrl && processedDataUrl.data && processedDataUrl.data['text/plain']) {
            let cleanUrl = processedDataUrl.data['text/plain'].replace(/^(.*)$/ , '$1');
            img.src = cleanUrl;
        }

        // Loop instantly using the next UI repaint trigger frame
        requestAnimationFrame(captureLoop);
    }

    captureLoop();
}
startStream();
""

# Run the live environment wrapper
display(js_stream)

```

File selection

```

# =====
# STEP 1: Dependencies & Model Download
# =====
!pip install ultralytics -U

```

```

!wget -O fire_and_smoke_yolov8.pt
https://github.com/Lefyd24/Forest-Fire-Smoke-Detection/raw/main/models/yolov8/weights/fire_model.pt

import os
import cv2
from ultralytics import YOLO
from google.colab import files # Used for native browser upload/download
widgets

# Load the fine-tuned fire and smoke model
model = YOLO('fire_and_smoke_yolov8.pt')

# =====
# STEP 2: Interactive Image Upload Widget
# =====
print("\n Please select your disaster/mission image to upload:")
uploaded = files.upload()

# Extract the filename dynamically from the upload dictionary
if not uploaded:
    print("❌ Upload cancelled.")
else:
    input_filename = list(uploaded.keys())[0]
    print(f"✅ Successfully uploaded: {input_filename}")

    # =====
    # STEP 3: YOLOv8 Inference Pipeline
    # =====
    print(f"[PROCESS] Running fire and smoke detection on {input_filename}...")

    # Run prediction pass (conf=0.35 matches your webcam script threshold)
    results = model.predict(source=input_filename, conf=0.35, imgsz=640, verbose=True)[0]

    # Generate the annotated canvas with bounding boxes drawn over
    # detections
    annotated_frame = results.plot()

```

```

    # Optional: Burn your custom state machine telemetry metadata onto the
    image for the presentation
    state_text = "STATE: CALAMITY_LOG | PERCEPTION: ACTIVE"
    cv2.putText(annotated_frame, state_text, (20, 40),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    # =====
    # STEP 4: Save & Trigger Automatic Download
    # =====
    output_filename = "detected_" + os.path.splitext(input_filename)[0] +
".jpg"
    cv2.imwrite(output_filename, annotated_frame)
    print(f"✅ Output image compiled as '{output_filename}'")

    # Trigger native browser download directly to your laptop's Download
    folder
    print("[DOWNLOAD] Sending processed image back to your system...")
    files.download(output_filename)

```

Human detection

```
# =====
```

STEP 1: Dependencies & Model Initialization

```
# =====
```

```
!pip install ultralytics -U
```

```
import os
```

```
import cv2
```

```
from ultralytics import YOLO
```

```
from google.colab import files # Used for native browser upload/download widgets
```

```
# Load the pre-trained nano model (optimized for fast embedded edge execution)
```

```
model = YOLO('yolov8n.pt')
```

```
# =====
```

STEP 2: Interactive Image Upload Widget

```
# =====
```

```
print("\n Please select your search-and-rescue target image to upload:")
```

```
uploaded = files.upload()
```

```
# Extract the filename dynamically from the upload dictionary
```

```
if not uploaded:
```

```

    print("❌ Upload cancelled.")
else:
    input_filename = list(uploaded.keys())[0]
    print(f"✅ Successfully uploaded: {input_filename}")

    # =====
    # STEP 3: YOLOv8 Inference Pipeline (Humans)
    # =====
    print(f"[PROCESS] Running human target detection on {input_filename}...")

    # Target strictly Class 0 (person) to filter out background objects and save computing
    # overhead
    results = model.predict(source=input_filename, classes=[0], conf=0.35, imgsz=640,
        verbose=True)[0]

    # Generate the annotated canvas with bounding boxes drawn over detections
    annotated_frame = results.plot()

    # Track survivor counts dynamically to output straight onto your presentation slide
    human_count = len(results.bboxes)

    # Burn your custom state machine telemetry metadata onto the image for the
    # presentation
    state_text = f"STATE: SEARCH_RESCUE | SURVIVORS LOCKED: {human_count}"
    cv2.putText(annotated_frame, state_text, (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
        (0, 255, 0), 2)

    # =====
    # STEP 4: Save & Trigger Automatic Download
    # =====
    output_filename = "detected_human_" + os.path.splitext(input_filename)[0] + ".jpg"
    cv2.imwrite(output_filename, annotated_frame)
    print(f"✅ Output image compiled as '{output_filename}'")

    # Trigger native browser download directly to your laptop's Download folder
    print("[DOWNLOAD] Sending processed image back to your system...")
    files.download(output_filename)

```